

Fase 6 — Rota de ingestão no gc-sistema · status de entrega

Sem número de bloco. Trilha da automação, lista "OCR Contratos (WhatsApp) — Breno" do ClickUp, numeração própria de 1 a 8. Ver docs/README.md e o CLAUDE.md .

Data: 2026-09-23 · Branch: feature-dev-breno-automacao · Plano: docs/automacao/migracao-telegram-integracao-automacao.md , Fase 6 e decisões 1, 9 a 12, 16.

Resumo

A rota está no repositório e as sete camadas do npm run validar passam (exit 0, em 2026-09-23, depois de a sprint 5 fechar). POST /api/ingestao/proposta grava proposta com itens, em rascunho, com histórico de origem, e vincula o documento.

Falta uma coisa para o fluxo do Telegram usar a rota: um endereço público do gc-sistema apontando para o gc-dev (seção 3). Até lá o Processar Documento segue gravando a proposta por REST direto, sem itens. A rota está pronta e testada por HTTP contra o gc-dev.

1. O que entrou no repositório

ARQUIVO	O QUÊ
src/lib/ingestao.ts	helper puro (regra) — novo
src/lib/ingestao.test.ts	21 casos de node --test — novo
src/app/api/ingestao/proposta/route.ts	route handler — novo
src/lib/supabase/middleware.ts	exceção de /api/ingestao/ (4 linhas)
scripts/validar-escrita.mjs	23 passos novos da ingestão, com limpeza
scripts/rotas-esperadas.txt	+ /api/ingestao/proposta (32 → 33), por --aceitar-rotas
.env.local (não versionado)	INGESTAO_TOKEN , 64 hex, gerado com openssl rand

Escrito antes em /Users/a1234/gc-sistema-staging/fase-6/ enquanto a sprint 5 estava ativa; copiado sem mudança depois que ela fechou.

src/lib/ingestao.ts — a regra

Reusa, sem duplicar: `resolverValorUnit`, `notaDeInferencia`, `normalizarUnidade`, `numeroItemDoDocumento`, `acrescentarObservacao` e `limparColunasGeradas` (`sprint 5`, `itens.ts`); `validarDesconto`, `validarSomaPct`, `pctToFraction` e `novaEntradaHistoricoProposta` (`propostas.ts`).

REGRA	ONDE
<code>valor_unit</code> ausente → inferido de total / quantidade, nota em <code>observacao</code> (decisão 10)	<code>montarItemIngestao</code>
Sem nenhum dos dois valores, ou quantidade nula/zero → recusa (decisão 10)	idem
Unidade → <code>QTD</code> / <code>M2</code> ; original não reconhecido em <code>observacao</code> (decisão 11)	idem
<code>numero</code> não inteiro → <code>null</code> , original em <code>observacao</code>	idem
<code>numero</code> repetido no documento → o segundo vira <code>null</code> , com nota (<code>unique</code> (<code>proposta_id</code> , <code>numero</code>))	<code>montarIngestao</code>
Nunca monta <code>valor_total</code> nem <code>area_m2</code> (colunas geradas)	<code>limparColunasGeradas</code>
Unitário × quantidade ≠ total impresso → grava o unitário, registra o total impresso	<code>montarItemIngestao</code>
Largura/altura acima de 10 lidas como milímetro → metros, original em <code>observacao</code>	<code>dimensaoEmMetros</code>
Um item sem conserto recusa a ingestão inteira — nada de proposta meio gravada	<code>montarIngestao</code>
Proposta nasce rascunho , <code>created_by</code> = profile de serviço	idem
Histórico desde o nascimento: <code>de</code> : <code>rascunho</code> , <code>para</code> : <code>rascunho</code> , <code>por</code> = uuid (decisão 12)	idem
Rastro do documento: <code>[automação · TELEGRAM · documento <id>]</code> em <code>observacao</code>	idem
Soma dos itens ≠ total do documento (>1%) → aviso , não recusa (o trigger da 5.6 faz o valor virar a soma)	idem
Token conferido em tempo constante	<code>tokenConfere</code>

Por que o rastro vai em `observacao`: a entrada de `historico` tem campos fixos (`de`, `para`, `em`, `por`, `motivo_rejeicao`, `detalhe_rejeicao`) e não comporta o `documentoId`. O vínculo forte continua sendo `documentos_processamento.proposta_criada_id`.

route.ts — a fronteira

POST `/api/ingestao/proposta`, header `x-ingestao-token`. Sem sessão não há RLS de usuário: a rota usa a service role e **confere ela mesma** documento × empresa e obra × empresa antes de gravar.

RESPOSTA	QUANDO
<code>201 { ok, propostaId, itens, avisos }</code>	criou proposta, itens, e vinculou o documento (<code>APROVADO</code> , <code>proposta_criada_id</code>)
<code>200 { ok, propostaId, jaExistia: true }</code>	o documento já tinha gerado proposta — o n8n reexecutou
<code>401</code>	token errado
<code>409</code>	número de proposta repetido na empresa (decisão 9), com a mensagem de <code>mensagemDeErroProposta</code>
<code>422</code>	payload recusado pela regra, documento ou obra de outra empresa, ou itens recusados pelo banco
<code>500</code>	rota sem configuração (<code>INGESTAO_TOKEN</code> , <code>INGESTAO_PROFILE_ID</code> , URL, service role)

Proposta e itens na mesma unidade: se o insert dos itens falhar, a proposta é **apagada** antes de responder. A rota também passa a fazer o vínculo do documento, que hoje é um nó separado no n8n — com ela, o `Processar Documento` troca dois nós (`Criar proposta` e `Documento aprovado`) por uma chamada.

`middleware.patch`

O middleware atual redireciona para `/login` toda requisição sem sessão, inclusive `/api/*` . A exceção deixa passar só o prefixo `/api/ingestao/` ; a proteção é o token.

2. Números reais — `npm run validar` , 2026-09-23, contra gc-dev

Comparados com os que a sprint 5 registrou no fechamento (`docs/sprint-5/5.8-status-entrega.md`).

CAMADA	FIM DA SPRINT 5	AGORA	DIFERENÇA
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint</code> warnings or errors (8s)	—
unitário	146 casos	167 casos, 167 pass, 0 falhas (3s)	+21 (<code>ingestao.test.ts</code>)
build	32 rotas	33 rotas, iguais à baseline (40s)	+1: <code>/api/ingestao/proposta</code>
runtime	47/47 rotas	47/47 rotas ok (23s)	— (rota fora, ver 3.1)
dados	18/18 checagens	18/18 checagens ok (2s)	— (a rota não lê nada que uma page não leia)

CAMADA	FIM DA SPRINT 5	AGORA	DIFERENÇA
escrita	122/122 passos	145/145 passos ok, nada sobrou em gc-dev (77s)	+23
navegador	57/57 passos	57/57 passos ok, console limpo (91s)	—

Os 23 passos novos da escrita, contra a rota rodando em `next start`:

- configuração: `INGESTAO_TOKEN` e `INGESTAO_PROFILE_ID` presentes;
- sem token → **401**, sem redirecionar para `/login` (prova a exceção do middleware); token errado → **401**;
- `pct > 100%` → **422**; obra que não é da empresa → **422**; item com quantidade zero e sem valor → **422** e a ingestão inteira recusada; e nenhuma dessas deixou proposta gravada;
- ingestão válida → **201** com 3 itens; proposta em `rascunho`, autor = profile de serviço; histórico `rascunho` → `rascunho` com `por` = uuid; `pct 30/70` gravados como fração; rastro do documento em `observacao`;
- itens: só com total → `valor_unit` 500 inferido e anotado; `UN` → `QTD`; `m²` → `M2` com 950×2100 mm convertidos para $0,95 \times 2,1$ m; `area_m2` e `valor_total` calculados pelo banco; valor da proposta = soma dos itens (**trigger da 5.6** agindo sobre a ingestão);
- documento vinculado: `APROVADO`, `proposta_criada_id` e obra;
- segunda chamada com o mesmo `documentoId` → **200**, mesmo id, sem segunda proposta;
- mesmo número com outro documento → **409** "Já existe uma proposta com esse número" (decisão 9);
- limpeza: proposta apagada com os itens, 2 documentos de teste apagados.

3. O que NÃO foi validado — pendências nominais

1. `/api/ingestao/proposta` não tem cobertura na camada runtime — de propósito, a saída 2 do plano. O `validar-runtime.mjs` exige de toda rota do `validacao-rotas.json` um `307` para `/login` sem sessão, e esta rota autentica por token e devolve 401. Ela é coberta pelas camadas build (monta) e escrita (23 passos por HTTP). A saída 1 (um `"esperaAnon": 401` no script) fica para quando houver a segunda rota de máquina (contrato, Seção 8).
2. O n8n ainda não chama a rota. Precisa de um endereço público do gc-sistema apontando para o gc-dev — o n8n está na nuvem e não alcança `localhost`, e o único deploy (Vercel) aponta para gc-prod e tem login da Vercel na frente. Decisão do Breno:
 - **deploy de preview na Vercel com as variáveis do gc-dev** e *Protection Bypass for Automation* — recomendado; exige push da branch, que só com pedido explícito;
 - túnel temporário do `next dev` durante os testes;
 - manter o REST direto (sem itens) até existir o ambiente.

3. Troca no **Processar Documento** de `Criar proposta` + `Documento aprovado` por um HTTP Request à rota, com o token numa Credential. Depende do item 2.
4. A tela não foi aberta com uma proposta da rota. A camada navegador cobre `/propostas` e a aba Itens com o seed; a proposta criada pela ingestão foi conferida por query e apagada.
5. Regra de milímetro é heurística (> 10 lê-se mm). Esquadria de mais de 10 m ou documento em centímetros é lido errado — o original fica em `observacao`.
6. Sem commit. Tudo no working tree, como o `CLAUDE.md` pede.

4. Itens gravados pelo n8n com a regra do sistema (decisão 21) — 23/09

A rota continua pronta e verde; o que mudou é que o n8n **não espera mais por ela** para gravar os itens.

4.1 O que o **Processar Documento** faz agora

```
Proposta aprovada? → Montar ingestão → Criar proposta → Tem itens?  
                                sim → Criar itens → Documento aprovado  
                                não → Documento aprovado  
  
Criar itens falhou → guarda o erro → APAGA a proposta → Registrar falha (revisão)  
Montar ingestão recusou (item sem conserto) → Registrar falha (revisão, não "falha técnica")  
  
Montar avisos → Preparar envio → Telegram - Enviar      (antes: sub-workflow Notificar)
```

- **Montar ingestão** roda `montarIngestao` de `src/lib/ingestao.ts`, empacotado com os helpers que ele usa. Proposta nasce **rascunho**, autor e histórico = profile de serviço, rastro do documento em `observacao`. Itens com `valor_unit` inferido, unidade mapeada, milímetro → metro, numeração do documento, **sem** `valor_total / area_m2`. O valor da proposta passa a ser a soma dos itens pelo trigger da 5.6.
- **Leitura não confiável** (número de item repetido ou soma divergente — o texto embaralhado da reserva Groq, fase 5 seção 5.3): a proposta entra **sem itens**, e o grupo recebe "itens NÃO gravados: leitura não confiável".
- **Proposta e itens na mesma unidade**: itens recusados pelo banco → a proposta é apagada.
- O aviso ao grupo diz quantos itens foram gravados.

4.2 Como regerar o **Montar ingestão**

```
node scripts/empacotar-ingestao-n8n.mjs > /tmp/ingestao-n8n.js
```

Cole no nó, substituindo tudo acima da linha `// ===== fim do código gerado =====`. A versão publicada foi gerada do commit `5b9f76f` (os `.ts` commitados). O `itens.ts` do working tree tem 60 linhas a mais da sprint 6 (bloco 6.4), só exportações novas — não muda a regra.

4.3 Números reais

VERIFICAÇÃO	RESULTADO
Pacote × código do repo, mesmas entradas	9/9 casos idênticos (inclui o PDF de 16 itens)
Código do <code>Montar ingestão</code> , offline, com a extração real do Qwen do <code>EB-25-08-0048</code>	10/10 — 16 itens, item 1 inferido, item 12 = $2 \times 1.986,08$, item 13 = $0,95 \times 2,1$ m, sem coluna gerada, leitura ruim → 0 itens, item sem conserto → recusa
Colunas da linha do item × tabela <code>itens</code>	as 16 existem, mais <code>proposta_id</code>
<code>n8n_validate_workflow</code>	0 erros, 0 warnings , 34 nós, 50 conexões
Referências a nós inexistentes, e ao <code>Notificar</code> removido	nenhuma
Webhook do bot depois da publicação	registrado, sem erro
Execuções do n8n por documento	2 → 1

4.4 O que NÃO foi validado

1. **Nada disso rodou no n8n.** O plano do n8n Cloud atingiu o limite de execuções ("Execution limit reached"). A proposta com itens gravados nas colunas foi provada offline e por `select` de colunas, **não** por uma execução real. Primeiro teste quando as execuções voltarem: a Vista Verde (4 itens) pelo bot.
2. **O rollback (itens recusados → proposta apagada) nunca disparou.**
3. **O `chat_id` do grupo está em dois nós** — `Preparar envio` (aqui) e `Resolver destino` (Notificar, que ficou só com o alerta de erro de sistema). A decisão 5 pedia um só; é o preço de tirar o sub-workflow do caminho. Os dois comentários avisam.
4. **Mensagens do grupo de admin ainda gastam execução.** O bot recebe toda mensagem do grupo (privacy mode desligado, decisão 5) e o workflow roda só para ignorá-la. O privacy desligado só era preciso para descobrir o `chat_id` do grupo, que já está anotado: para **enviar** ao grupo ele não é necessário. Religar o privacy no `@obraminds_ingestao_v2_bot` corta essas execuções — decisão do Breno, porque contraria o texto da decisão 5.